Arquitectura lógica del sistema de control de rescates y adopciones

Análisis UML, patrones de diseño y flujo Gitflow

Actividad de Ingeniería de Software

Documento técnico de diseño

07/10/2026

Alcance. Diseño lógico de referencia para una plataforma que registra rescates, coordina la atención veterinaria, gestiona postulaciones y formaliza adopciones con trazabilidad.

Contenido

1 1. Resumen ejecutivo 1
2 2. Análisis del escenario 1
2.1 2.1. Problema y alcance 1
2.2 2.2. Actores 1
2.3 2.3. Requerimientos funcionales 2
2.4 2.4. Requerimientos no funcionales 2
3 3. Modelo UML de casos de uso 2
3.1 3.1. Reglas de negocio principales 3
4 4. Modelo UML de clases 4
4.1 4.1. Estados relevantes 6
5 5. Arquitectura lógica 6
6 6. Patrones de diseño aplicados 8
6.1 6.1. Factory Method — creacional 8
6.2 6.2. Observer — comportamiento 8
6.3 6.3. Strategy — comportamiento 9
6.4 6.4. Repository — estructural / acceso a datos 9
7 7. Datos, seguridad y trazabilidad 9
8 8. Flujo Gitflow y evidencia de avances 9
8.1 8.1. Convención de ramas 9
8.2 8.2. Registro mínimo recomendado 10
8.3 8.3. Enlace público de GitHub 10
9 9. Plan de pruebas y criterios de aceptación 10
10 10. Conclusiones 11
11 Anexo A. Matriz breve de trazabilidad 11

Nota sobre el caso. Como la guía técnica de la Unidad 1 no fue adjuntada en esta sesión, el documento explicita supuestos funcionales razonables del caso “Control de Rescate”. Deben contrastarse con la guía y la rúbrica institucional antes de entregar.

1 1. Resumen ejecutivo

Se propone una arquitectura lógica modular para un sistema de control de rescates y adopciones de animales. La solución cubre el ciclo completo: reporte ciudadano o institucional, validación y priorización, asignación de un equipo, atención veterinaria, publicación para adopción, evaluación de postulantes, formalización contractual y seguimiento posterior.

La propuesta separa presentación, aplicación, dominio e infraestructura. Esta separación permite cambiar la interfaz web o móvil sin alterar las reglas del negocio, sustituir el proveedor de notificaciones y probar los casos de uso con dobles de prueba. El diseño aplica cuatro patrones: **Factory Method** para crear notificaciones, **Observer** para publicar eventos de dominio, **Strategy** para priorizar rescates y **Repository** como abstracción de persistencia. También se define Gitflow para que el avance del diseño quede auditable en GitHub.

2 2. Análisis del escenario

2.1 2.1. Problema y alcance

Las organizaciones de rescate suelen administrar información dispersa en llamadas, hojas de cálculo y mensajes. Esto dificulta conocer el estado de un rescate, evitar duplicidades, demostrar la atención recibida y seleccionar adoptantes con criterios consistentes. El sistema propuesto centraliza la información y conserva una línea de tiempo de cada animal.

Incluye: registro de reportes; geolocalización textual; priorización; asignación de recursos; evidencias fotográficas; ficha veterinaria; publicación; postulaciones; evaluación; contrato; notificaciones; auditoría y reportes.

No incluye en esta primera versión: pagos, diagnóstico veterinario automatizado, reconocimiento facial, integración con autoridades externas ni decisiones irreversibles sin intervención humana.

2.2 2.2. Actores

Actor	Responsabilidad	Interacción principal
Ciudadano / reportante	Informa un animal en riesgo y aporta ubicación, descripción y evidencia.	Registrar reporte
Rescatista	Atiende el caso, registra evidencias y actualiza la operación.	Asignación y atención
Coordinador	Valida, prioriza, asigna y supervisa el ciclo de rescate y adopción.	Gestión operativa
Veterinario	Registra valoración, diagnóstico, tratamiento y aptitud para adopción.	Ficha clínica
Adoptante	Consulta animales, presenta una postulación y firma el acuerdo.	Postulación y adopción
Administrador	Mantiene usuarios, roles, catálogos y parámetros.	Administración

Servicio externo	Correo, SMS o push para avisos transaccionales.	Notificaciones

2.3 2.3. Requerimientos funcionales

1. **RF-01.** Registrar un reporte con ubicación, fecha, descripción, especie estimada, fotografías y datos de contacto.
2. **RF-02.** Validar el reporte, detectar duplicados y asignar una prioridad basada en riesgo, urgencia y disponibilidad.
3. **RF-03.** Crear una operación de rescate y asignar uno o más integrantes con recursos y hora estimada.
4. **RF-04.** Registrar llegada, evidencias, traslado, estado del animal y cierre de la operación.
5. **RF-05.** Registrar la evaluación veterinaria y las restricciones de manejo o adopción.
6. **RF-06.** Publicar una ficha de adopción únicamente cuando el animal sea apto según las reglas configuradas.
7. **RF-07.** Recibir, consultar y evaluar postulaciones con trazabilidad de observaciones y decisiones.
8. **RF-08.** Formalizar la adopción mediante un acuerdo versionado y notificar a las partes.
9. **RF-09.** Consultar indicadores por estado, tiempo de respuesta, especie, zona y resultado.
10. **RF-10.** Administrar usuarios, roles, estados y parámetros sin modificar el historial ya cerrado.

2.4 2.4. Requerimientos no funcionales

1. **Seguridad:** autenticación, autorización por rol, cifrado en tránsito, mínimo privilegio y registro de auditoría.
2. **Disponibilidad:** operación tolerante a fallas parciales de correo; las notificaciones deben reintentarse.
3. **Rendimiento:** consultas de listados con paginación y respuesta objetivo menor a 2 segundos bajo carga normal.
4. **Mantenibilidad:** módulos cohesivos, interfaces explícitas, pruebas unitarias de reglas y documentación UML versionada.
5. **Privacidad:** limitar la exposición de datos del adoptante y conservar evidencias con control de acceso.
6. **Trazabilidad:** toda transición relevante debe registrar quién, cuándo, qué cambió y por qué.

3 3. Modelo UML de casos de uso

El diagrama identifica las fronteras del sistema y las relaciones de inclusión. La notificación se modela como una capacidad transversal: no sustituye la decisión humana, pero informa los cambios de estado que ya fueron autorizados.

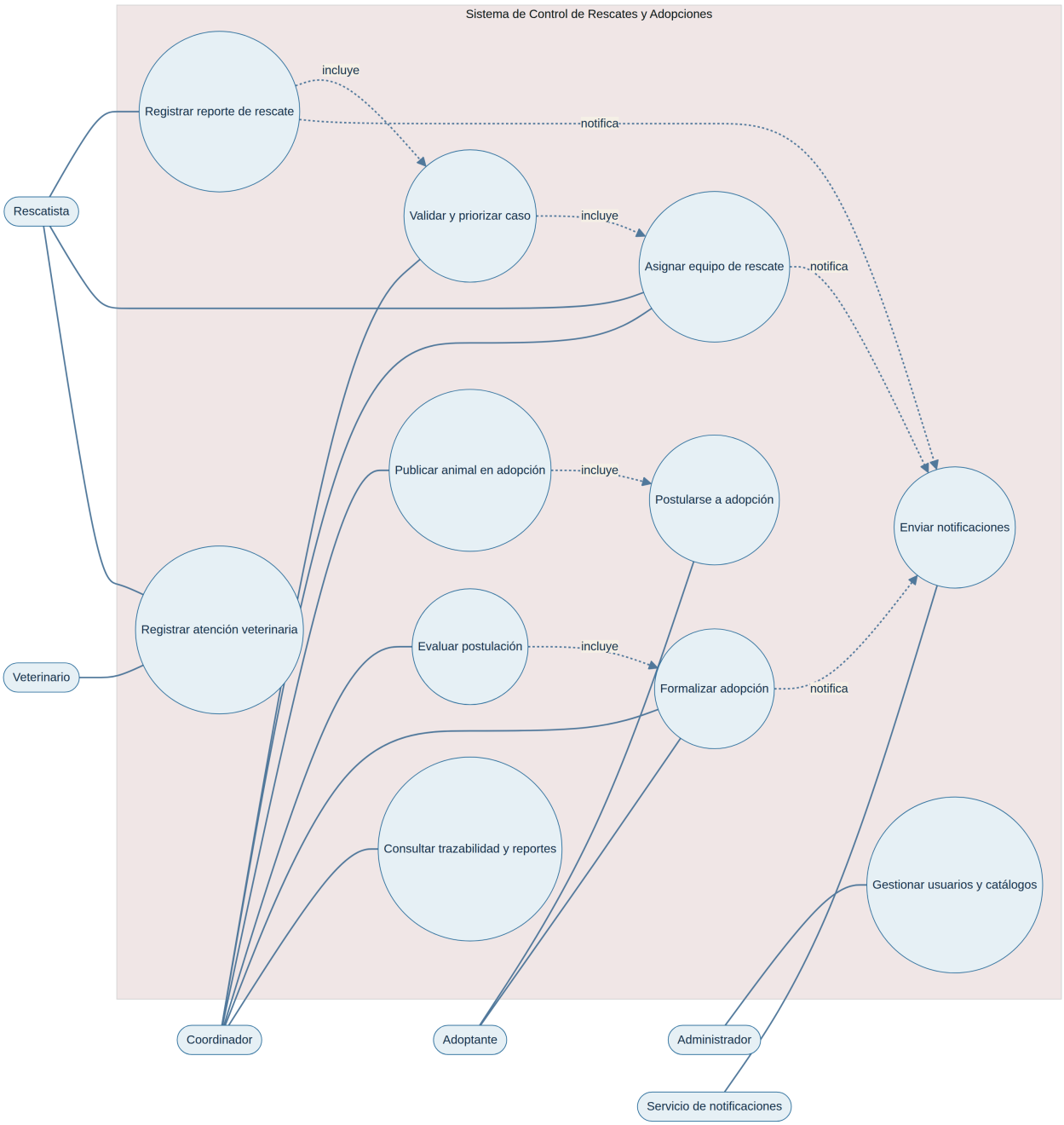


图 1　Diagrama de casos de uso del sistema.

3.1 3.1. Reglas de negocio principales

1. Un reporte no puede pasar a **asignado** sin ubicación, prioridad y responsable.
2. Una operación no puede cerrarse si no existe evidencia mínima de resultado y ubicación de destino.
3. Un animal no puede publicarse si la evaluación veterinaria lo marca como no apto o si existe una cuarentena vigente.
4. Una postulación solo puede aprobarse cuando el adoptante está verificado y la ficha sigue disponible.
5. La formalización de una adopción cambia el estado del animal a **adoptado** y cierra las postulaciones incompatibles.
6. Las transiciones son auditables; borrar físicamente una entidad histórica no está permitido.

4 4. Modelo UML de clases

El modelo de clases combina entidades del dominio y servicios de aplicación. Las entidades conservan identidad y reglas; los servicios orquestan casos de uso; las dependencias hacia repositorios y notificaciones se expresan mediante interfaces, reduciendo el acoplamiento con la infraestructura.

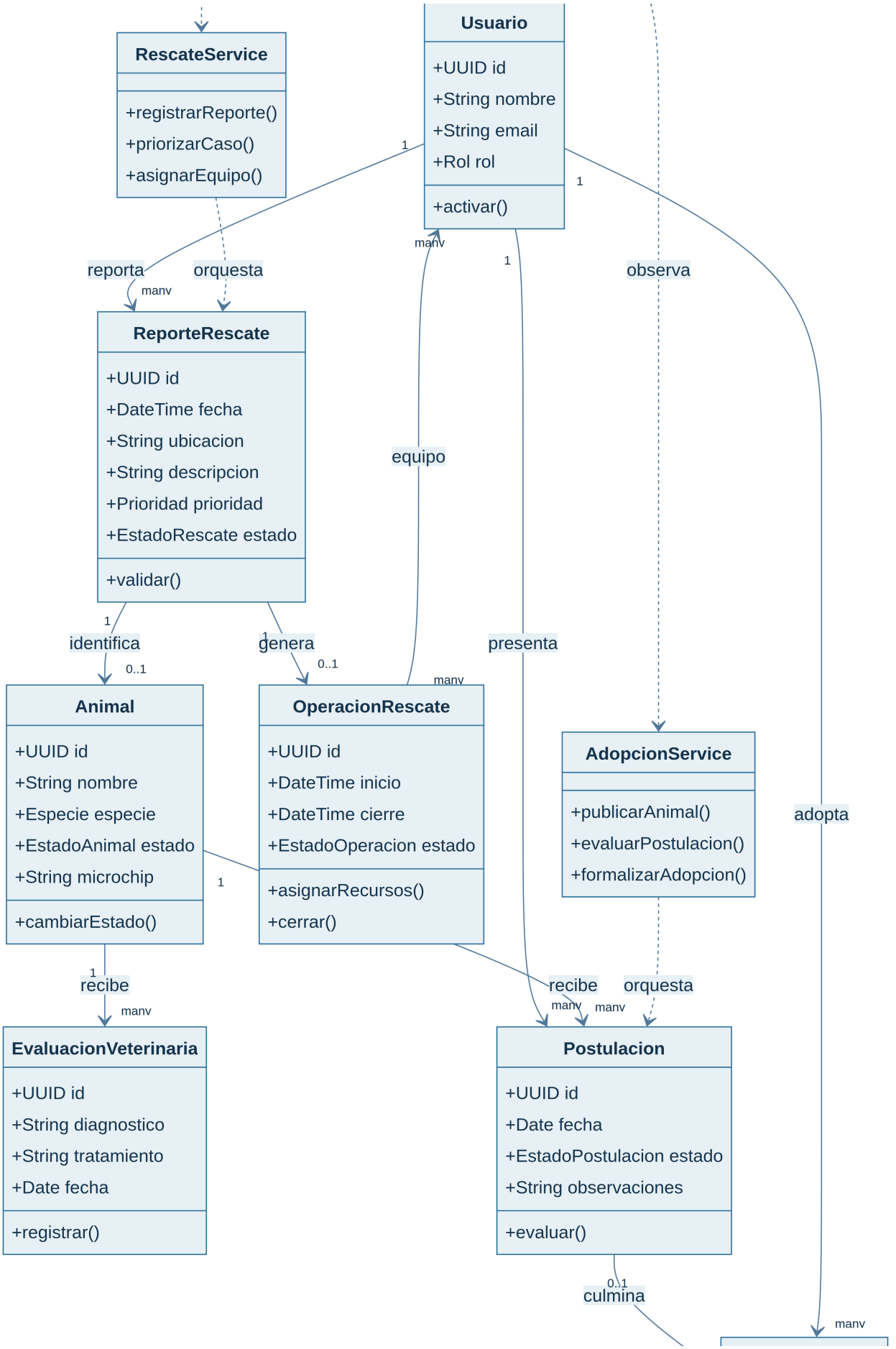

RescateService
+registrarReporte()
+priorizarCaso()
+asignarEquipo()
Usuario
+UUID id
+String nombre
+String email
+Rol rol
+activar()
reporta
orquesta
observa
ReporteRescate
+UUID id
+DateTime fecha
+String ubicacion
+String descripcion
+Prioridad prioridad
+EstadoRescate estado
+validar()
equipo
identifica
genera
presenta
Animal
+UUID id
+String nombre
+Especie especie
+EstadoAnimal estado
+String microchip
+cambiarEstado()
OperacionRescate
+UUID id
+DateTime inicio
+DateTime cierre
+EstadoOperacion estado
+asignarRecursos()
+cerrar()
AdopcionService
+publicarAnimal()
+evaluarPostulacion()
+formalizarAdopcion()
adopta
recibe
orquesta
EvaluacionVeterinaria
+UUID id
+String diagnostico
+String tratamiento
+Date fecha
+registrar()
Postulacion
+UUID id
+Date fecha
+EstadoPostulacion estado
+String observaciones
+evaluar()
culmina
1
manv
0..1

图 2　Clases de dominio, servicios y relaciones principales.

4.1 4.1. Estados relevantes

Objeto	Ciclo de vida resumido
Reporte	Borrador ⟶ Validado ⟶ Priorizado ⟶ Asignado ⟶ Cerrado / Cancelado
Animal	Reportado ⟶ En rescate ⟶ En recuperación ⟶ Disponible ⟶ En proceso de adopción ⟶ Adoptado
Postulación	Recibida ⟶ En evaluación ⟶ Aprobada / Rechazada / Retirada
Adopción	Preparada ⟶ Firmada ⟶ Seguimiento ⟶ Cerrada

5 5. Arquitectura lógica

La arquitectura propuesta es una variante de arquitectura en capas con inversión de dependencias. La capa de dominio no conoce la base de datos ni el proveedor de mensajes. Los controladores reciben solicitudes, los servicios coordinan transacciones y los adaptadores implementan puertos definidos por el dominio.

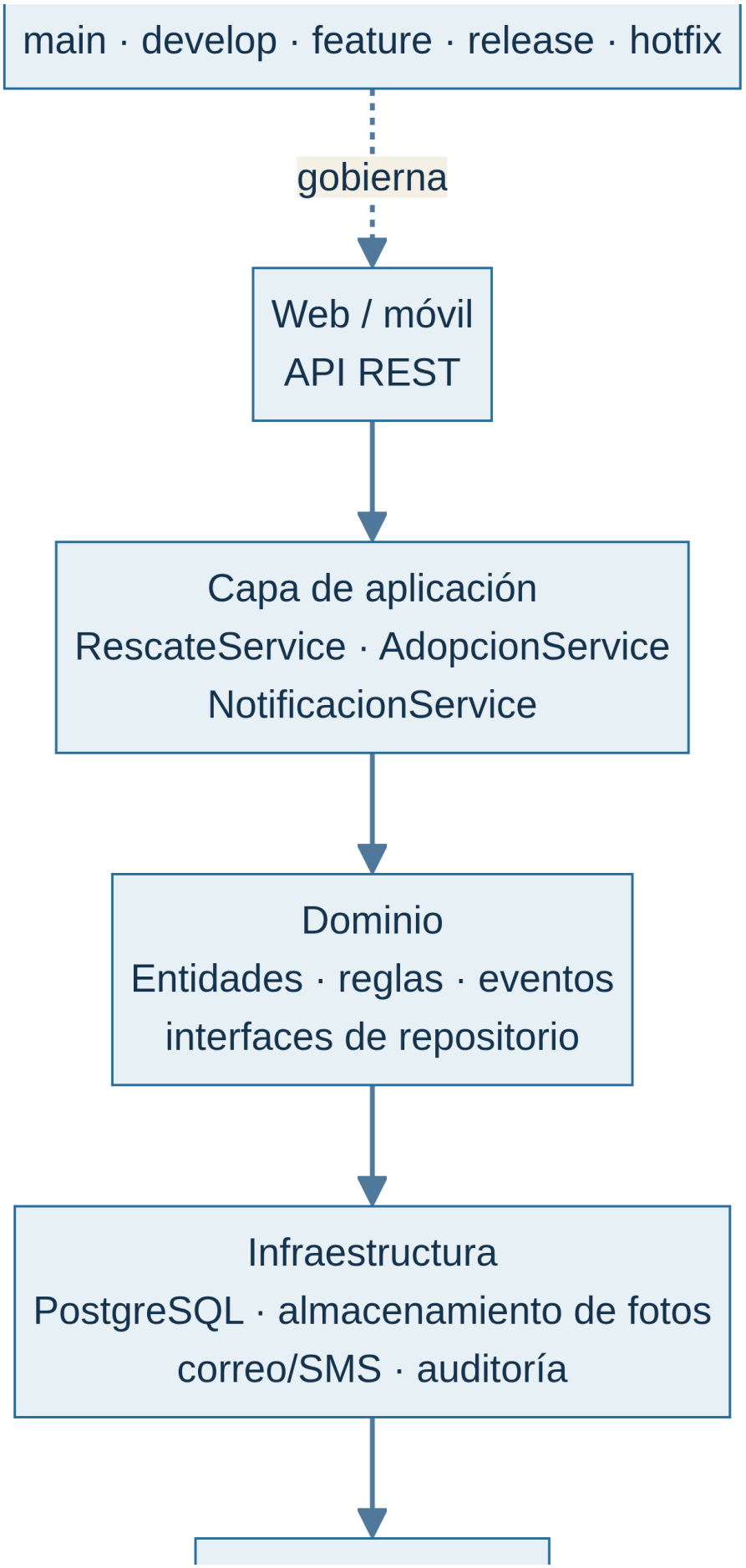

main · develop · feature · release · hotfix
gobierna
Web / móvil
API REST
Capa de aplicación
RescateService · AdopcionService
NotificacionService
Dominio
Entidades · reglas · eventos
interfaces de repositorio
Infraestructura
PostgreSQL · almacenamiento de fotos
correo/SMS · auditoría

图 3 Arquitectura por capas y gobierno del cambio mediante Gitflow.

Capa	Responsabilidad	Ejemplos
Presentación	Exponer API, validar formato y manejar autenticación.	REST, web, móvil
Aplicación	Coordinar transacciones y permisos de cada caso de uso.	RescateService, AdopcionService
Dominio	Proteger reglas, estados, eventos y contratos.	Animal, ReporteRescate, políticas
Infraestructura	Persistir, almacenar evidencias y conectar proveedores.	PostgreSQL, S3, correo/SMS

6 6. Patrones de diseño aplicados

6.1 6.1. Factory Method — creacional

El sistema no debe conocer en cada caso de uso si una notificación se enviará por correo, SMS o push. `NotificacionFactory` crea el producto adecuado a partir de una preferencia o configuración. Para agregar un canal se incorpora una nueva implementación sin modificar la lógica de rescate.

```
interface Notificacion { enviar(destino, mensaje) }
class EmailNotificacion implements Notificacion
class SmsNotificacion implements Notificacion
class PushNotificacion implements Notificacion
class NotificacionFactory {
  crear(tipo) -> Notificacion
}
```

Beneficio: encapsula la creación y evita condicionales repetidos. **Riesgo:** una fábrica demasiado grande puede convertirse en un punto de acoplamiento; se mitiga separando fábricas por familia de productos cuando crezca el sistema.

6.2 6.2. Observer — comportamiento

Los servicios publican eventos como `RescateAsignado`, `AnimalPublicado` o `AdopcionFormalizada`. Observadores independientes reaccionan para notificar, auditar o actualizar métricas. El emisor no necesita conocer cada consumidor.

```
subject: EventBus
observers: NotificacionHandler, AuditoriaHandler, MetricasHandler
subject.publish(AnimalPublicado)
```

Beneficio: desacopla efectos secundarios y permite reintentos. **Riesgo:** el orden y la eventual consistencia deben documentarse; los eventos críticos requieren identificador idempotente.

6.3 6.3. Strategy — comportamiento

La priorización de un rescate puede cambiar por política institucional. `PrioridadStrategy` define el contrato y permite elegir `RiesgoVitalStrategy`, `CercaniaStrategy` o `DisponibilidadStrategy` sin modificar el servicio que consume el resultado.

```
interface PrioridadStrategy { calcular(reporte, contexto) -> Prioridad }
class RiesgoVitalStrategy implements PrioridadStrategy
class DisponibilidadStrategy implements PrioridadStrategy
```

Beneficio: hace explícitas las políticas variables y facilita pruebas comparativas. La estrategia activa debe quedar registrada para explicar por qué un caso recibió una prioridad.

6.4 6.4. Repository — estructural / acceso a datos

`AnimalRepository`, `ReporteRepository` y `AdopcionRepository` definen operaciones orientadas al dominio. Adaptadores concretos implementan esas interfaces con PostgreSQL u otra persistencia. Así, las pruebas pueden usar repositorios en memoria y la capa de dominio permanece independiente.
Beneficio: reduce el acoplamiento y protege la evolución de la base de datos. **Criterio:** no convertir el repositorio en una segunda capa de reglas; las invariantes deben vivir en entidades o servicios de dominio.

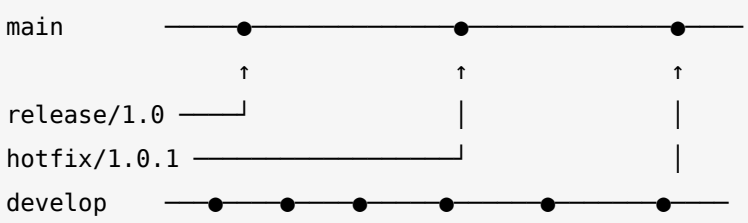
7 7. Datos, seguridad y trazabilidad

La clave primaria de cada entidad será un UUID. Las fotografías se almacenarán fuera de la base de datos y se referenciarán mediante una URL protegida y con expiración. La base conservará metadatos, estados y hashes de evidencia. Los datos personales del adoptante se mostrarán solo a roles autorizados y se separarán de los datos públicos de la ficha animal.
La auditoría debe registrar usuario, rol, fecha, entidad, operación, valores anteriores y posteriores, motivo y correlación de solicitud. Los archivos de evidencia deberán pasar validación de tipo, tamaño y malware antes de ser visibles. El cierre de una adopción requerirá autorización del coordinador y generará una versión inmutable del acuerdo.

8 8. Flujo Gitflow y evidencia de avances

8.1 8.1. Convención de ramas

El repositorio debe mantener `main` como rama estable y publicable, y `develop` como integración. Cada avance se realiza en una rama `feature/*`; una entrega se prepara en `release/*` y una corrección urgente parte de `hotfix/*`. Las ramas temporales se integran mediante pull request y revisión.



```
              ↑     ↑     ↑
feature/uml  feature/patrones  feature/documentacion
```

8.2 8.2. Registro mínimo recomendado

1. `feature/analisis-requerimientos`: actores, alcance y requerimientos.
2. `feature/uml-casos-clases`: fuentes Mermaid y exportaciones de diagramas.
3. `feature/patrones-diseno`: decisiones de Factory Method, Observer, Strategy y Repository.
4. `feature/gitflow-documentacion`: README, guía de ramas y evidencias.
5. `release/1.0.0`: revisión final del PDF y checklist de la rúbrica.
6. `hotfix/*`: solo correcciones críticas posteriores a una versión publicada.

Comandos de referencia:

```
git init
git checkout -b develop
git checkout -b feature/uml-casos-clases
git add . && git commit -m "docs: agregar modelos UML"
git checkout develop && git merge --no-ff feature/uml-casos-clases
git checkout -b release/1.0.0
git checkout main && git merge --no-ff release/1.0.0
git tag -a v1.0.0 -m "Entrega 1.0.0"
```

8.3 8.3. Enlace público de GitHub

El documento queda preparado para publicar en un repositorio público con el nombre sugerido `control-rescate-adopciones`. En esta sesión no existe una cuenta GitHub iniciada, por lo que no se debe inventar una URL. Tras crear el repositorio, sustituya el marcador por el enlace real:

> **Enlace público:** `https://github.com/dayanaponcei2002/control-rescate-adopciones`

Para publicar las ramas y etiquetas:

```
git remote add origin https://github.com/dayanaponcei2002/control-rescate-
adopciones.git
git push -u origin main develop --tags
git push origin --all
```

La evidencia debe mostrar en GitHub las ramas `main`, `develop`, al menos dos ramas `feature/*`, una `release/*` y, si aplica, una `hotfix/*`, junto con commits descriptivos y pull requests integrados.

9 9. Plan de pruebas y criterios de aceptación

Prueba	Escenario	Resultado esperado
P-01	Reporte sin ubicación	Rechazo con mensaje y sin cambio de estado

P-02	Dos reportes de la misma zona y animal	Alerta de posible duplicado para revisión
P-03	Animal con cuarentena vigente	Publicación impedida
P-04	Adopción aprobada	Contrato versionado, estado actualizado y avisos emitidos
P-05	Falla del proveedor de correo	Evento persistido y reintento sin duplicar aviso
P-06	Usuario sin rol coordinador	Acceso denegado y evento de auditoría

La aceptación de la entrega requiere: diagramas legibles; requerimientos trazables a casos de uso; al menos tres patrones explicados con intención, estructura y beneficio; repositorio con Gitflow visible; README con instrucciones; y PDF con el enlace público real actualizado.

10 10. Conclusiones

La solución propuesta convierte un proceso operativo sensible en un flujo auditable y extensible. UML proporciona un lenguaje común para actores, responsabilidades y estructura; los patrones reducen acoplamientos en los puntos que probablemente evolucionen; y Gitflow organiza el trabajo para que cada avance pueda revisarse y revertirse.

Antes de la entrega académica deben validarse los supuestos contra la guía de la Unidad 1, reemplazarse el marcador de GitHub por la URL pública real y adjuntarse capturas o enlaces de las ramas y pull requests. Con esas verificaciones, el diseño ofrece una base mantenible para una posterior implementación incremental.

11 Anexo A. Matriz breve de trazabilidad

Requisito	**Caso de uso**	**Elemento de diseño**
RF-01–RF-04	Registrar y coordinar rescate	ReporteRescate, OperacionRescate, Strategy
RF-05–RF-06	Atender y publicar animal	EvaluacionVeterinaria, Animal, Repository
RF-07–RF-08	Evaluar y formalizar adopción	Postulacion, Adopcion, Observer
RF-09–RF-10	Reportes y administración	AuditoriaHandler, roles, servicios